データフレームの作成、加工、出力

医療データ科学実習

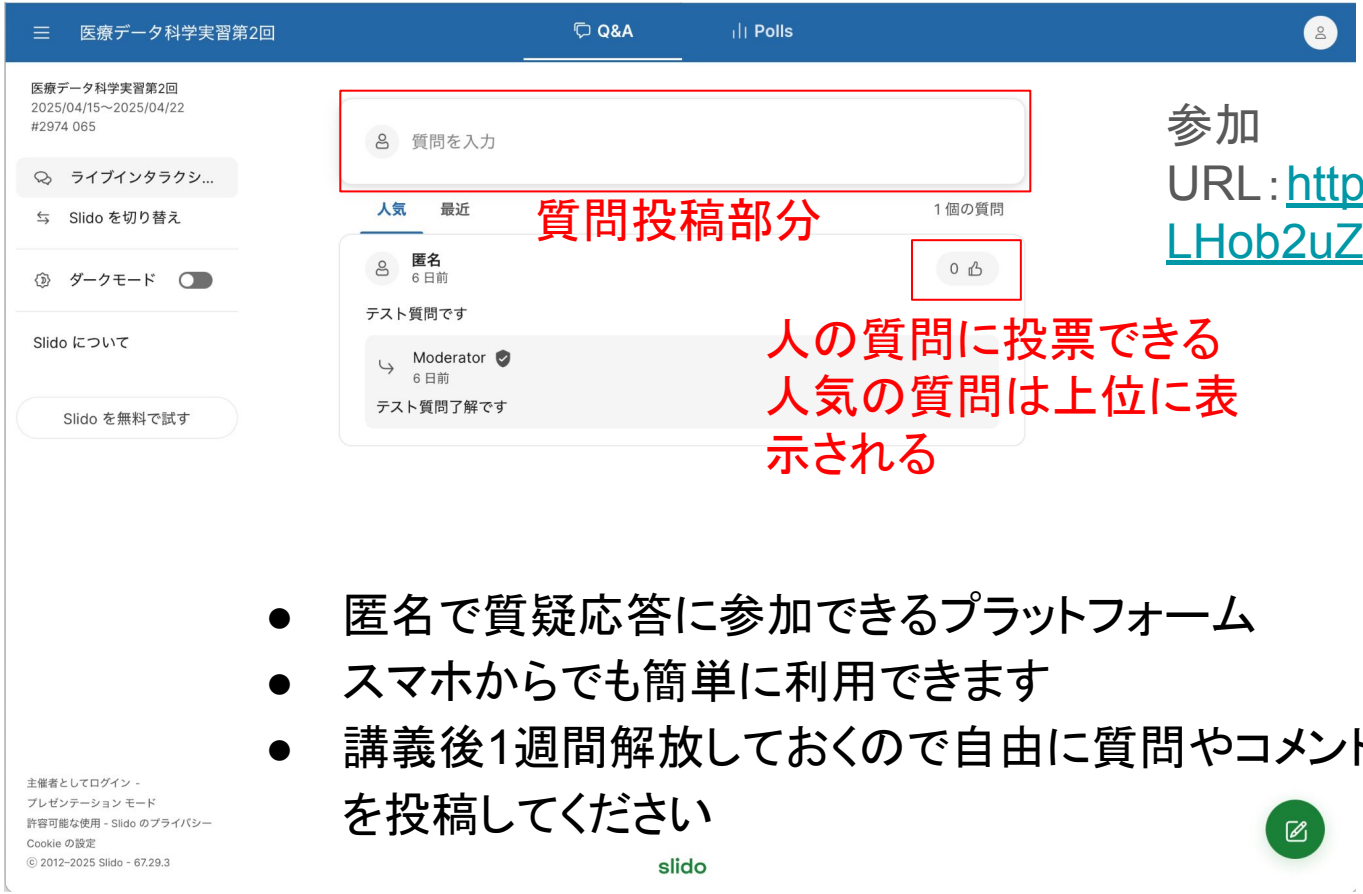
（Practice of Biomedical Data Science）

第3回 2026年4月28日

臨床統計学

emoto.ryo.5m@kyoto-u.ac.jp

Slidoで質疑応答に参加しよう



質問投稿部分

人の質問に投票できる
人気の質問は上位に表示される

参加
URL：https://app.sli.do/event/1dsLHob2uZXnaPLAq3GkbG

- 匿名で質疑応答に参加できるプラットフォーム
- スマホからでも簡単に利用できます
- 講義後1週間解放しておくので自由に質問やコメントを投稿してください

Chap0. 本日の到達目標

表形式の医療データをRで自在に扱う

```
> infert
    education age parity induced case spontaneous stratum pooled.stratum
1      0-5yrs  26      6       1    1           2       1              3
2      0-5yrs  42      1       1    1           0       2              1
3      0-5yrs  39      6       2    1           0       3              4
4      0-5yrs  34      4       2    1           0       4              2
5     6-11yrs  35      3       1    1           1       5             32
6     6-11yrs  36      4       2    1           1       6             36
```

- 多くのデータは数値と文字の混ざった表形式

目標

1. 複数のオブジェクトを格納できるリスト (list) の基本操作をマスターする
2. 表形式のデータフレーム (data.frame) の基本操作をマスターする
3. データセットの外部ファイルへの出力方法をマスターする

Chap1. リスト

- リストとは
- リストの作成
- リストの操作

リスト（list）とは

- Rのあらゆるオブジェクトをまとめて格納できる構造
  - ベクトル，行列，そして別のリストでさえも，1つのリストにまとめることができる

- 異なるデータ型・長さのデータを格納できる
  - 第2回で学んだベクトル，行列の中身の属性は全て同じである必要があったが
    リストは数値と文字をまとめて一つのオブジェクトに格納できる
  - データ数（長さ）が異なっていても良い

リストの作成1

- リストは `list()` 関数によって作成できる
  - 例1: 直接データを入れて作成

```
> l1 <- list(10, "a", 1:10, c("apple", "banana", "cherry"))
> l1
[[1]]
[1] 10

[[2]]
[1] "a"

[[3]]
 [1]  1  2  3  4  5  6  7  8  9 10

[[4]]
[1] "apple"  "banana" "cherry"
```

リストの作成1

- リストは `list()` 関数によって作成できる
  - 例1: 直接データを入れて作成

```
> l1 <- list(10, "a", 1:10, c("apple", "banana", "cherry"))
> l1
[[1]]
[1] 10

[[2]]
[1] "a"

[[3]]
 [1]  1  2  3  4  5  6  7  8  9 10

[[4]]
[1] "apple"  "banana" "cherry"
```

ベクトル同様 “,” で区切る

二重鉤括弧 [[ ]] の中にリストの要素番号が表示される

リストの作成2

- リストは `list()` 関数によって作成できる
  - 例2: 存在するオブジェクトを入れて作成

```
> x <- c(10, 20, 30, 40, 50)
> y <- 1:10
> fruits <- c("apple", "banana", "cherry", "date", "elderberry")
> balloon <- fruits %in% c("apple", "banana")
> A <- matrix(1:12, nrow = 3, ncol = 4)
>
> l2 <- list(x, y, fruits, balloon, A)
> l2
[[1]]
[1] 10 20 30 40 50

[[2]]
 [1]  1  2  3  4  5  6  7  8  9 10
…
```

リストの作成2

- リストは `list()` 関数によって作成できる
  - 例2: 存在するオブジェクトを入れて作成

```
> x <- c(10, 20, 30, 40, 50)
> y <- 1:10
> fruits <- c("apple", "banana", "cherry", "date", "elderberry")
> balloon <- fruits %in% c("apple", "banana")
> A <- matrix(1:12, nrow = 3, ncol = 4)
>
> l2 <- list(x, y, fruits, balloon, A)
> l2
[[1]]
[1] 10 20 30 40 50

[[2]]
 [1]  1  2  3  4  5  6  7  8  9 10
…
```

オブジェクト名を “,” で区切る

リストの作成3

- リストは `list()` 関数によって作成できる
  - 例3: 各要素に名前をつけて作成

```
> l3 <- list(vec1 = x, vec2 = y, vec3 = fruits, vec4 = balloon, mat1 = A)
> l3
$vec1
[1] 10 20 30 40 50

$vec2
 [1]  1  2  3  4  5  6  7  8  9 10

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"

$vec4
[1]  TRUE  TRUE FALSE FALSE FALSE

$mat1
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

リストの作成3

- リストは `list()` 関数によって作成できる
  - 例3: 各要素に名前をつけて作成

```
> l3 <- list(vec1 = x, vec2 = y, vec3 = fruits, vec4 = balloon, mat1 = A)
> l3
$vec1
[1] 10 20 30 40 50

$vec2
 [1]  1  2  3  4  5  6  7  8  9 10

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"

$vec4
[1]  TRUE  TRUE FALSE FALSE FALSE

$mat1
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

要素名=オブジェクト名で指定

ドルマーク $ の後にリストの要素名が表示される

リストの属性1

- リストオブジェクトにも長さや名前などの属性が存在
- `str()` 関数はオブジェクトの構造をまとめて表示

```
> length(l3)        # 全要素数
[1] 5
> names(l3)         # 要素名
[1] "vec1" "vec2" "vec3" "vec4" "mat1"
> str(l3)           # 全体の構造を表示
List of 5
 $ vec1: num [1:5] 10 20 30 40 50
 $ vec2: int [1:10] 1 2 3 4 5 6 7 8 9 10
 $ vec3: chr [1:5] "apple" "banana" "cherry" "date" ...
 $ vec4: logi [1:5] TRUE TRUE FALSE FALSE FALSE
 $ mat1: int [1:3, 1:4] 1 2 3 4 5 6 7 8 9 10 ...
```

リストの長さ

リストの各要素の名前と型

リストの属性2

- `str()` 関数は大きなリストで便利

```
> l4 <- list(list1 = l1, list3 = l3)
> str(l4)
List of 2
 $ list1:List of 4
  ..$ : num 10
  ..$ : chr "a"
  ..$ : int [1:10] 1 2 3 4 5 6 7 8 9 10
  ..$ : chr [1:3] "apple" "banana" "cherry"
 $ list3:List of 5
  ..$ vec1: num [1:5] 10 20 30 40 50
  ..$ vec2: int [1:10] 1 2 3 4 5 6 7 8 9 10
  ..$ vec3: chr [1:5] "apple" "banana" "cherry" "date" ...
  ..$ vec4: logi [1:5] TRUE TRUE FALSE FALSE FALSE
  ..$ mat1: int [1:3, 1:4] 1 2 3 4 5 6 7 8 9 10 ...
```

リストのリストも作成可能

l4 は要素数2のリスト

各要素の構造も表示

要素の名前がない時は空欄

リストの要素の抽出

- `[[ ]]` による抽出: 要素番号もしくは名前で指定

```
> l3[[3]]
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
> l3[["vec3"]]
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

- `$` **による抽出 : 要素名で指定**

```
> l3$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

（発展）リストのインデックス操作1

- `[ ]` による操作により対応する要素を持つ**リスト**を作成できる
  - 正の整数インデックス

```
> l3[1] #正の整数インデックス
$vec1
[1] 10 20 30 40 50
> l3[c(1,3)] #複数の指定
$vec1
[1] 10 20 30 40 50

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

  - 負の整数インデックス

```
> l3[-c(2,4,5)] #負の整数インデックス
$vec1
[1] 10 20 30 40 50

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

（発展）リストのインデックス操作2

- `[ ]` による操作により対応する要素を持つ**リスト**を作成できる
  - 論理値インデックス

```
> l3[c(TRUE,FALSE,TRUE,FALSE,FALSE)] #論理値インデックス
$vec1
[1] 10 20 30 40 50

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

  - 名前インデックス

```
> l3[c("vec1","vec3")] # 名前インデックス
$vec1
[1] 10 20 30 40 50

$vec3
[1] "apple"      "banana"     "cherry"     "date"       "elderberry"
```

Chap2. データフレーム

- データフレームとは
- データフレームの作成
- データフレームの操作

データフレーム（data.frame）とは

- 表形式のデータを扱うための構造
- 要素が全て同じ長さのベクトルである特別なリスト（list）
  - 長さが同じであれば変数の型は異なってもよい
    文字ベクトル、数値ベクトル、論理値ベクトルなど

    →リストと同じ操作が可能

- 行が個人（sample）、列が変数を表す
  - 基本的には各行が患者や観測データを表す
  - 列の要素は同じ順番で対応付け

    →行列と似た操作が可能

データフレームの作成

- データフレームは `data.frame()` 関数によって作成できる
  - リスト同様に名前を付けて作成可能
    →名前は列名になる

```
> df1 <- data.frame( ID = 1:2, Treatment = c("A", "B") )
> df1
  ID Treatment
1  1         A
2  2         B

> x1 = 1:5
> x2 = c("Control", "Treatment", "Treatment", "Control", "Treatment")
> x3 = c(45, 62, 38, 55, 71)
> df2 <- data.frame(ID = x1, Group = x2, Age = x3)
> str(df2)
'data.frame':  5 obs. of  3 variables:
 $ ID   : int  1 2 3 4 5
 $ Group: chr  "Control" "Treatment" "Treatment" "Control" ...
 $ Age  : num  45 62 38 55 71
```

データフレームの属性1

- データフレームの大きさ
  - `nrow()`: 行方向の大きさ
  - `ncol()`: 列方向の大きさ
  - `dim()`: 行、列の大きさを同時に確認

```
> nrow(df2)
[1] 5
> ncol(df2)
[1] 3
> dim(df2)
[1] 5 3
```

データフレームの属性1

- データフレームの大きさ
  - `nrow()`: 行方向の大きさ
  - `ncol()`: 列方向の大きさ
  - `dim()`: 行、列の大きさを同時に確認

```
> nrow(df2)
[1] 5
> ncol(df2)
[1] 3
> dim(df2)
[1] 5 3
```

5行3列、行数、列数の順

データフレームの属性2

- 行方向、列方向の名前
  - `rownames()`: 行の名前（指定しないと行番号）
  - `colnames(), names()`: 列の名前

```
> rownames(df2)
[1] "1" "2" "3" "4" "5"
> colnames(df2)
[1] "ID"    "Group" "Age"
> names(df2)
[1] "ID"    "Group" "Age"
```

データフレームの要素の抽出1: リスト形式

- リストと同様の方法による抽出
  - `$`: **名前で抽出**
  - `[[ ]]`: 列番号（要素番号）で抽出
  - `[]`: 複数の列を同時に抽出

```
> df2$Age
[1] 45 62 38 55 71
> df2[[3]]
[1] 45 62 38 55 71
> df2[c(2,3)]
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
```

データフレームの要素の抽出1: リスト形式

- リストと同様の方法による抽出
  - `$`: **名前で抽出**
  - `[[ ]]`: 列番号（要素番号）で抽出
  - `[]`: 複数の列を同時に抽出

```
> df2$Age
[1] 45 62 38 55 71
> df2[[3]]
[1] 45 62 38 55 71
> df2[c(2,3)]
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
```

リストとしての抽出方法

データフレームの要素の抽出2: 行列形式

- 行列と同様の方法による抽出

```
> df2[1, ] # 1 行目全体
  ID   Group Age
1  1 Control  45
> df2[, 2] # 2 列目全体
[1] "Control"   "Treatment" "Treatment" "Control"   "Treatment"
> df2[, -1] # 1 列目以外
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
> df2[1:2, 2:3] # 表の一部
      Group Age
1   Control  45
2 Treatment  62
```

データフレームの要素の抽出2: 行列形式

- 行列と同様の方法による抽出

```
> df2[1, ] # 1 行目全体
  ID   Group Age
1  1 Control  45
> df2[, 2] # 2 列目全体
[1] "Control"   "Treatment" "Treatment" "Control"   "Treatment"
> df2[, -1] # 1 列目以外
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
> df2[1:2, 2:3] # 表の一部
      Group Age
1   Control  45
2 Treatment  62
```

データフレームとして抽出される

ベクトルとして抽出される

データフレームの要素の抽出2: 行列形式

```
> df2[2, 3, drop=FALSE] #データフレームのまま保持
  Age
2  62

> #論理インデックス
> df2[df2$Age>60,] #各行に対する論理インデックス
  ID     Group Age
2  2 Treatment  62
5  5 Treatment  71
> df2[,colnames(df2) %in% c("Group", "Age")] #各列に対する論理インデックス
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
```

データフレームの要素の抽出2: 行列形式

```
> df2[2, 3, drop=FALSE] #データフレームのまま保持
  Age
2  62

> #論理インデックス
> df2[df2$Age>60,] #各行に対する論理インデックス
  ID     Group Age
2  2 Treatment  62
5  5 Treatment  71
> df2[,colnames(df2) %in% c("Group", "Age")] #各列に対する論理インデックス
      Group Age
1   Control  45
2 Treatment  62
3 Treatment  38
4   Control  55
5 Treatment  71
```

よく使う方法

データフレームへの列の追加

- 列指定による結合（列の追加）
  - `$`: 新しい列名を指定してベクトルを代入

```
> # 追加したい新しいベクトル（5人分の体重）
> x4 <- c(60, 75, 52, 68, 80)
>
> df2$Weight <- x4
> df2
  ID     Group Age Weight
1  1   Control  45     60
2  2 Treatment  62     75
3  3 Treatment  38     52
4  4   Control  55     68
5  5 Treatment  71     80
```

データフレームへの列の追加

- 列指定による結合（列の追加）
  - `$`: 新しい列名を指定してベクトルを代入

```
> # 追加したい新しいベクトル（5人分の体重）
> x4 <- c(60, 75, 52, 68, 80)
>
> df2$Weight <- x4
> df2
  ID     Group Age Weight
1  1   Control  45     60
2  2 Treatment  62     75
3  3 Treatment  38     52
4  4   Control  55     68
5  5 Treatment  71     80
```

df2にまだ存在しない名前にする必要がある

データフレームの結合1

- 列方向の結合
  - `cbind()`: 行列と同様

```
> y1 <- c("No", "Yes", "No", "No", "Yes") # 再発の有無
> y2 <- c(60, 24, 48, 55, 12) # 追跡期間/生存期間（月）
> df_outcome <- data.frame(Recurrence = y1, Survival_Months = y2)
>
> # cbind() で 横に結合
> df3 <- cbind(df2, df_outcome)
> df3
  ID     Group Age Weight Recurrence Survival_Months
1  1   Control  45     60         No              60
2  2 Treatment  62     75        Yes              24
3  3 Treatment  38     52         No              48
4  4   Control  55     68         No              55
5  5 Treatment  71     80        Yes              12
```

データフレームの結合2

- 列方向の結合
  - `merge()`: キーとなる変数を指定して列方向に結合

```
> ID <- c(1,2,3,6,7)
> y1 <- c("No", "Yes", "No", "No", "Yes")
> y2 <- c(60, 24, 48, 55, 12)
> df_outcome2 <- data.frame(ID = ID, Recurrence = y1, Survival_Months = y2)
>
> df3_merge <- merge(df2, df_outcome2, by = "ID", all = TRUE)
> df3_merge
  ID     Group Age Weight Recurrence Survival_Months
1  1   Control  45     60         No              60
2  2 Treatment  62     75        Yes              24
3  3 Treatment  38     52         No              48
4  4   Control  55     68       <NA>              NA
5  5 Treatment  71     80       <NA>              NA
6  6      <NA>  NA     NA         No              55
7  7      <NA>  NA     NA        Yes              12
```

データフレームの結合2

- 列方向の結合
  - `merge()`: キーとなる変数を指定して列方向に結合

```
> ID <- c(1,2,3,6,7)
> y1 <- c("No", "Yes", "No", "No", "Yes")
> y2 <- c(60, 24, 48, 55, 12)
> df_outcome2 <- data.frame(ID = ID, Recurrence = y1, Survival_Months = y2)
>
> df3_merge <- merge(df2, df_outcome2, by = "ID", all = TRUE)
> df3_merge
  ID     Group Age Weight Recurrence Survival_Months
1  1   Control  45     60         No              60
2  2 Treatment  62     75        Yes              24
3  3 Treatment  38     52         No              48
4  4   Control  55     68       <NA>              NA
5  5 Treatment  71     80       <NA>              NA
6  6      <NA>  NA     NA         No              55
7  7      <NA>  NA     NA        Yes              12
```

結合したいデータセットを二つ指定

キーとなる変数名の指定

all = FALSE: 共通するデータのみを結合（デフォルト）
all = TRUE: 共通しないデータも残す

データフレームの結合3

- 行方向の結合
  - `rbind()`: 行列と同様

```
> # 別施設のデータ
> df_siteB <- data.frame(
+   ID = 6:8,
+   Group = c("Treatment", "Control", "Treatment"),
+   Age = c(52, 47, 65),
+   Weight = c(70, 62, 78)
+ )
>
> # rbind() で 施設A(df2) と 施設B(df_siteB) を縦に結合
> df4 <- rbind(df2, df_siteB)
> df4
  ID     Group Age Weight
1  1   Control  45     60
2  2 Treatment  62     75
3  3 Treatment  38     52
4  4   Control  55     68
5  5 Treatment  71     80
6  6 Treatment  52     70
7  7   Control  47     62
8  8 Treatment  65     78
```

データフレームの確認1

- `str()`: リストとしての内容確認
- `head()`: 6行目まで (default) のデータの表示

```
> str(df4)
'data.frame':    8 obs. of  4 variables:
 $ ID    : int  1 2 3 4 5 6 7 8
 $ Group : chr  "Control" "Treatment" "Treatment" "Control" ...
 $ Age   : num  45 62 38 55 71 52 47 65
 $ Weight: num  60 75 52 68 80 70 62 78
> head(df4)
  ID     Group Age Weight
1  1   Control  45     60
2  2 Treatment  62     75
3  3 Treatment  38     52
4  4   Control  55     68
5  5 Treatment  71     80
6  6 Treatment  52     70
```

データフレームの確認1

- `str()`: リストとしての内容確認
- `head()`: 6行目まで (default) のデータの表示

```
> str(df4)
'data.frame':    8 obs. of  4 variables:
 $ ID    : int  1 2 3 4 5 6 7 8
 $ Group : chr  "Control" "Treatment" "Treatment" "Control" ...
 $ Age   : num  45 62 38 55 71 52 47 65
 $ Weight: num  60 75 52 68 80 70 62 78
> head(df4)
  ID     Group Age Weight
1  1   Control  45     60
2  2 Treatment  62     75
3  3 Treatment  38     52
4  4   Control  55     68
5  5 Treatment  71     80
6  6 Treatment  52     70
```

n = ...の指定で先頭から何行表示するかを指定できる

tail()関数も同様の関数で後ろから表示する関数

データフレームの確認2

- `summary()`: 各変数の要約統計量の確認

```
> summary(df4)
       ID          Group                Age            Weight
 Min.   :1.00   Length:8           Min.   :38.00   Min.   :52.00
 1st Qu.:2.75   Class :character   1st Qu.:46.50   1st Qu.:61.50
 Median :4.50   Mode  :character   Median :53.50   Median :69.00
 Mean   :4.50                      Mean   :54.38   Mean   :68.12
 3rd Qu.:6.25                      3rd Qu.:62.75   3rd Qu.:75.75
 Max.   :8.00                      Max.   :71.00   Max.   :80.00
```

  - 数値に対しては最小値、第一四分位点、中央値、平均、第三四分位点、最大値
  - 文字に対しては長さ、class、mode
  - factorに対しては頻度

データフレームのソート

- 行に対するベクトルを用いた抽出を利用
  - `order()`: ベクトルの昇順の要素番号を返す関数
  - `[]`: 要素番号のベクトルを用いて行に対して抽出

```
> sort_ind = order(df2$Age)
> sort_ind
[1] 3 1 4 2 5
>
> df2_sort = df2[sort_ind,]
> df2_sort
  ID     Group Age Weight
3  3 Treatment  38     52
1  1   Control  45     60
4  4   Control  55     68
2  2 Treatment  62     75
5  5 Treatment  71     80
```

データフレームのソート

- 行に対するベクトルを用いた抽出を利用
  - `order()`: ベクトルの昇順の要素番号を返す関数
  - `[]`: 要素番号のベクトルを用いて行に対して抽出

```
> sort_ind = order(df2$Age)
> sort_ind
[1] 3 1 4 2 5
>
> df2_sort = df2[sort_ind,]
> df2_sort
  ID     Group Age Weight
3  3 Treatment  38     52
1  1   Control  45     60
4  4   Control  55     68
2  2 Treatment  62     75
5  5 Treatment  71     80
```

df2$Ageを小さい方から並べた時の元の要素番号

正の数値ベクトルによるインデックス操作

データフレームの変数のclassの確認と変更

- `class()`: 特定の列のデータ型の確認に使用
- `as.○○○()`: ○○○型に変更（numeric, character, factorなど）
  - 今回の例ではcharacter型をfactor型に変更

```
> class(df4$Group)
[1] "character"
> df4$Group <- as.factor(df4$Group)
> summary(df4)
       ID            Group        Age            Weight
 Min.   :1.00   Control  :3   Min.   :38.00   Min.   :52.00
 1st Qu.:2.75   Treatment:5   1st Qu.:46.50   1st Qu.:61.50
 Median :4.50                 Median :53.50   Median :69.00
 Mean   :4.50                 Mean   :54.38   Mean   :68.12
 3rd Qu.:6.25                 3rd Qu.:62.75   3rd Qu.:75.75
 Max.   :8.00                 Max.   :71.00   Max.   :80.00
```

factor型になったことで頻度が集計されている

（発展）ファクター (factor) 型の詳細1

- カテゴリカルデータを効率的に扱うためのデータ型
- 見た目は文字型に近いが、内部的には整数値ベクトルと対応する文字列の組

```
> df4$Group
[1] Control   Treatment Treatment Control   Treatment Treatment Control
[8] Treatment
Levels: Control Treatment
> attributes(df4$Group)
$levels
[1] "Control"   "Treatment"

$class
[1] "factor"

> str(df4$Group)
 Factor w/ 2 levels "Control","Treatment": 1 2 2 1 2 2 1 2
```

出力は文字列に似ているが引用符なし

カテゴリの種類が表示される

オブジェクトの属性を全て表示する関数
→ levels属性にカテゴリの種類が格納

中身は数値ベクトル

1が”Control”(=levels[1]), 2が”Treatment”(=levels[2])

（発展）ファクター (factor) 型の詳細2

- `factor()`: `factor` 型のオブジェクトを作成する関数
  - 引数 `levels`: カテゴリの設定
  - 引数 `labels`: 表示されるカテゴリ名の設定
  - (引数 `ordered`: 順序付きカテゴリかどうか(TRUE/FALSE))

```
> df2$Group
[1] "Control"   "Treatment" "Treatment" "Control"   "Treatment"
>
> fac1 <- factor(df2$Group, levels = c("Treatment", "Control"))
> fac1
[1] Control   Treatment Treatment Control   Treatment
Levels: Treatment Control
> str(fac1)
 Factor w/ 2 levels "Treatment","Control": 2 1 1 2 1
>
> fac2 <- factor(df2$Group, levels = c("Treatment", "Control"), labels = c("治療群", "コントロール群"))
> fac2
[1] コントロール群 治療群         治療群         コントロール群 治療群
Levels: 治療群 コントロール群
> str(fac2)
 Factor w/ 2 levels "治療群","コントロール群": 2 1 1 2 1
```

文字列ベクトル

levelsの順序を逆にしたので中身の数値ベクトルも1, 2が逆転
→参照カテゴリの設定などに使用

表示名の変更（日本語化）

連続変数からカテゴリ変数への変更

- `ifelse()`: 条件を設定し、条件に合う場合と合わない場合の処理を行う
  - 引数1: 条件
  - 引数2: 条件に合う場合の処理
  - 引数3: 条件に合わない場合の処理

```
> Age_category = ifelse(df4$Age<65, "<65", ">=65")
> Age_category
[1] "<65"  "<65"  "<65"  "<65"  ">=65" "<65"  "<65"  ">=65"
>
> df4$Age_cat <- factor(Age_category)
> summary(df4)
       ID             Group        Age            Weight      Age_cat
 Min.   :1.00   Control  :3   Min.   :38.00   Min.   :52.00   <65 :6
 1st Qu.:2.75   Treatment:5   1st Qu.:46.50   1st Qu.:61.50   >=65:2
 Median :4.50                 Median :53.50   Median :69.00
 Mean   :4.50                 Mean   :54.38   Mean   :68.12
 3rd Qu.:6.25                 3rd Qu.:62.75   3rd Qu.:75.75
 Max.   :8.00                 Max.   :71.00   Max.   :80.00
```

連続変数からカテゴリ変数への変更

- `ifelse()`: 条件を設定し、条件に合う場合と合わない場合の処理を行う
  - 引数1: 条件
  - 引数2: 条件に合う場合の処理
  - 引数3: 条件に合わない場合の処理

```
> Age_category = ifelse(df4$Age<65, "<65", ">=65")
> Age_category
[1] "<65"  "<65"  "<65"  "<65"  ">=65" "<65"  "<65"  ">=65"
>
> df4$Age_cat <- factor(Age_category)
> summary(df4)
       ID             Group        Age            Weight      Age_cat
 Min.   :1.00   Control  :3   Min.   :38.00   Min.   :52.00   <65 :6
 1st Qu.:2.75   Treatment:5   1st Qu.:46.50   1st Qu.:61.50   >=65:2
 Median :4.50                 Median :53.50   Median :69.00
 Mean   :4.50                 Mean   :54.38   Mean   :68.12
 3rd Qu.:6.25                 3rd Qu.:62.75   3rd Qu.:75.75
 Max.   :8.00                 Max.   :71.00   Max.   :80.00
```

df4$Age<65がTRUEならば“<65”を代入
FALSEならば”>=65”を代入

Chap3. ファイルの出力

- データフレームの出力
- 図の出力

データフレームの出力1: 作業ディレクトリの確認、変更

- `getwd()`: Rstudioを実行しているディレクトリ（フォルダの住所）を確認
- `setwd()`: 作業ディレクトリを変更
  - メニューから操作も可能
    1. RStudioの上部メニュー [Session] をクリック
    2. [Set Working Directory] を選択
    3. [Choose Directory...] で、保存したいフォルダを選ぶ（ない場合は作ることも可能）
  - 例：”/Users/ryoemoto/Documents/医療データ科学実習 /第3回”に保存したい場合

```
> # 作業ディレクトリを変更
> setwd("/Users/ryoemoto/Documents/医療データ科学実習 /第3回")
> # 確認
> getwd()
[1] "/Users/ryoemoto/Documents/医療データ科学実習 /第3回"
```

データフレームの出力2: write.csv

- `write.csv()`: 行列、データフレームをcsvファイルで出力する関数
  - `x`: オブジェクト
  - `file`: 出力ファイル名
  - `row.names`: 行の名前を出力するかどうかの指定

```
> write.csv(x = df4, file = "testdata1.csv")
```

→エクセル等で確認すると列名が表示されている

```
> write.csv(x = df4, file = "testdata2.csv", row.names = FALSE)
```

→行名の表示をオフに

※`write.table()`関数: `write.csv()`の元になっている関数で
区切り文字の設定なども可能

図の出力1

- RStudioメニューからの書き出し
  1. RStudioの右下 [Plots] パネルに図が表示されている状態で、[Export] をクリック
  2. 用途に合わせて選択：
     - Save as Image... ：PNGやJPEG形式（スライドや報告書用）
     - Save as PDF... ：PDF形式
  3. [Directory] が先ほど設定した「作業ディレクトリ」になっているか確認し、
     サイズを設定し、File nameを入力して[Save] を押す

図の出力2

- 関数を用いた出力
  - `pdf()`関数, `png()`関数
    1. `pdf()`関数で出力ファイルの設定
    2. 図の作成コード
    3. `dev.off()`関数で保存
  - `cairo_pdf()`:`pdf()`関数と同様の関数、日本語に対応（オススメ）

第2回の演習(p. 57)

- Rstudioメニューからの出力する場合のコード例

```
library(ggplot2)
library(showtext)
showtext_auto()

# 乱数シードを固定して再現性を確保
set.seed(123)

n <- 1000; mu <- 50; sigma <- 10
data <- data.frame(value = rnorm(n, mean = mu, sd = sigma))

# ggplot2 でヒストグラムを可視化
ggplot(data, aes(x = value)) +
  geom_histogram(binwidth = 2, fill = 'skyblue', color = 'black') +
  labs(
    title = '正規分布 N(50, 10^2) に基づく擬似データ',
    x = '値',
    y = '頻度'
  ) +
  theme_minimal()
```

第2回の演習(p. 57)

- 関数を用いて出力する場合のコード例

```
# ggplot2 オブジェクトを作成
g <- ggplot(data, aes(x = value)) +
  geom_histogram(binwidth = 2, fill = 'skyblue', color = 'black') +
  labs(
    title = '正規分布 N(50, 10^2) に基づく擬似データ',
    x = '値',
    y = '頻度'
  ) +
  theme_minimal()

# 出力ファイルの設定
cairo_pdf("hist.pdf",width=7,height=7)
# 画像をPlot windowに出力するコード（複数行でも可）
print(g)
# ここまでの出力を保存
dev.off()
```

Rのマニュアルの使い方

- `help()`関数: Rの関数や組み込みデータセットに関してのマニュアル表示
  - 関数の名前を入れて使用→右下のHelpウィンドウにマニュアルが表示される
    `help(関数名)`
  - 下記のショートコマンドが存在
    `?関数名`

```
help(write.csv)
?write.csv
```

- パッケージ自体のマニュアルはCRANに存在（Webで検索すれば見つかる）
  - 該当のパッケージに含まれる関数についてのマニュアルが全て見れる

Chap4. 演習問題

演習問題1

1. 下記の仮想的な健診データをデータフレーム`df_kenshin`として作成してください

ID	年齢	身長(cm)	体重(kg)
01_001	45	170	65
01_002	52	160	55
01_003	68	155	60
01_004	74	180	85
01_005	39	165	70

2. BMIを計算し、作成したデータフレームに列`BMI`を追加してください
   BMI = [体重 (kg)] / [身長 (m)の二乗]
3. 関数`ifelse()`を使って、BMIが25以上の人を "Overweight", 25未満の人を "Normal" とする新しい因子型(factor)の列`BMI_cat`を追加してください
4. 関数`summary()`を使って、作成したデータフレームを確認してください

演習問題2

1. Rの組み込みデータフレーム `infert` のマニュアルを参照し、
   どのようなデータか確認してください
2. 関数 `summary()` を使って、データフレーム `infert` の内容を確認してください
3. 下記の列のみに限定したデータフレーム`df_infert1` を作成してください
   1 Education
   2 age
   3 parity
   4 number of prior induced abortions
   5 case status
   6 number of priorspontaneous abortions
4. マニュアルのFormatの欄の記述を参考に、各変数を連続変数、カテゴリカル変数のどちらとして扱うべきか判定し、適切に変換したデータフレーム`df_infert2` を作成してください
5. 関数 `summary()` を使って、作成したデータフレームを確認してください

演習問題3

1. 演習問題2で作成したデータから、不妊症 (Infertility) である症例を抜き出したデータフレーム `df_infert3` を作成してください
   ※`case status`が`case`の症例が不妊症 (Infertility) である症例

2. 関数 `dim()` もしくは `nrow()` を使って抜き出した症例数を確認してください

3. データフレーム `df_infert3` を年齢の昇順に並べたデータフレーム`df_infert3_sort` を作成してください

4. 関数 `head()` を使ってソートしたデータの先頭の行を確認してください

演習問題1: コード例

```
> df_kenshin <- data.frame(
+   ID = c("01_001","01_002","01_003","01_004","01_005"),
+   Age = c(45, 52, 68, 74, 39),
+   Height = c(170, 160, 155, 180, 165), # cm
+   Weight = c(65, 55, 60, 85, 70)       # kg
+ )
>
> BMI = df_kenshin$Weight / (df_kenshin$Height/100)^2
>
> df_kenshin$BMI = BMI
> df_kenshin
      ID Age Height Weight      BMI
1 01_001  45    170     65 22.49135
2 01_002  52    160     55 21.48437
3 01_003  68    155     60 24.97399
4 01_004  74    180     85 26.23457
5 01_005  39    165     70 25.71166
```

演習問題2: コード例

```
> ?infert
> summary(infert)
     出力は省略

> df_infert1 <- infert[,1:6]
> df_infert2 <- df_infert1
> df_infert2$induced <- as.factor(df_infert1$induced)
> df_infert2$case <- as.factor(df_infert1$case)
> df_infert2$spontaneous <- as.factor(df_infert1$spontaneous)
> summary(df_infert2)
   education        age            parity      induced case    spontaneous
 0-5yrs : 12   Min.   :21.00   Min.   :1.000   0:143   0:165   0:141
 6-11yrs:120   1st Qu.:28.00   1st Qu.:1.000   1: 68   1: 83   1: 71
 12+ yrs:116   Median :31.00   Median :2.000   2: 37           2: 36
               Mean   :31.50   Mean   :2.093
               3rd Qu.:35.25   3rd Qu.:3.000
               Max.   :44.00   Max.   :6.000
```

演習問題3: コード例

```
> df_infert3 <- df_infert2[df_infert2$case == "1", ]
> dim(df_infert3)
[1] 83  6
> df_infert3_sort <- df_infert3[order(df_infert3$age),]
> head(df_infert3_sort)
   education age parity induced case spontaneous
9    6-11yrs  21      1       0    1           1
67   12+ yrs  21      1       0    1           1
7    6-11yrs  23      1       0    1           0
83   12+ yrs  23      1       0    1           1
51   12+ yrs  24      3       1    1           2
19   6-11yrs  25      3       2    1           1
```

提出課題

演習問題3で作成したデータをcsvファイルに出力して提出してください

提出方法: KULMSの課題提出機能

提出期限: 4/29(水) 23:55